

Institut Spécialisé de Technologie Appliquée Kasba Tadla

RAPPORT DE STAGE

**Conception et Réalisation d'une Application Web De
Gestion des Réclamations**

Filière: Développement Web Full Stack



Encadrant professionnel :
OUHSSINI Ahmed

Encadrant pédagogique :
RACHID Siham

Réalise par :
CHAIBI Abdellah

Période : Du 2 mars au 28 mars

Année :2025/2026

Remerciements

Au terme de ce stage, il m'est particulièrement agréable d'exprimer ma profonde gratitude et mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation et au bon déroulement de cette expérience professionnelle.

Je tiens à adresser mes remerciements les plus chaleureux à mes encadrants professionnels, **M. Ouhsini Ahmed** et **M. Mazine Hamza**, pour leur accompagnement, leur disponibilité ainsi que pour la qualité de leurs conseils et orientations tout au long de ce stage. Leur encadrement dans le domaine du développement web m'a permis de mieux comprendre le fonctionnement d'un environnement professionnel et d'améliorer mes compétences techniques et pratiques.

J'exprime également ma reconnaissance à mes formateurs, **Mme Rachid Siham** et **M. Khafouri Mohamed**, pour la qualité de la formation dispensée, ainsi que pour leur suivi académique et leurs conseils tout au long de mon parcours. Leurs enseignements m'ont permis d'acquérir les bases solides nécessaires pour réussir ce stage.

Je tiens aussi à remercier chaleureusement l'ensemble des **autres formateurs** pour leurs efforts et leur engagement dans la transmission des connaissances, ainsi que **l'équipe administrative** pour son soutien constant, sa disponibilité et son accompagnement bienveillant.

Enfin, j'adresse mes sincères remerciements à **ma famille** pour son soutien inconditionnel, sa patience et ses encouragements constants, ainsi qu'à **mes collègues** pour leur motivation et leur présence tout au long de mon parcours.

Résumé

Dans un contexte marqué par la transformation digitale et l'évolution des besoins des usagers, le projet « **Réclamation** » a été conçu afin de proposer une solution numérique dédiée à la gestion des réclamations.

Il s'agit d'une application web permettant aux utilisateurs de soumettre, suivre et consulter leurs réclamations de manière simple et structurée. L'objectif principal est de faciliter l'accès au service et d'améliorer l'expérience utilisateur à travers une interface claire et fonctionnelle.

Par ailleurs, cette plateforme permet de centraliser et d'optimiser le traitement des réclamations au niveau administratif, garantissant ainsi un meilleur suivi et une gestion plus efficace des demandes.

Ce projet s'inscrit dans une démarche de digitalisation des services administratifs, visant à améliorer la communication avec les usagers, réduire les délais de traitement et renforcer l'efficacité du système.

Sur le plan technique, la solution repose sur une architecture basée sur la séparation des couches (frontend, backend et base de données), ce qui assure une meilleure maintenabilité, évolutivité et performance.

Une attention particulière a été accordée à l'ergonomie, à la sécurité des données et à la stabilité du système, afin de garantir une solution fiable et adaptée aux besoins des utilisateurs.

Table des Abréviations

Abréviation	Signification
API	Application Programming Interface (Interface de Programmation d'Application)
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer)
UI	User Interface (Interface Utilisateur)
UX	User Experience (Expérience Utilisateur)
SPA	Single Page Application (Application Monopage)
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
MVC	Model – View – Controller (Modèle – Vue – Contrôleur)
UML	Unified Modeling Language (Langage de Modélisation Unifié)
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
SQL	Structured Query Language
IDE	Integrated Development Environment (Environnement de Développement Intégré)
REST	Representational State Transfer
Auth	Authentication (Authentification)
JWT	JSON Web Token
CSS	Cascading Style Sheets
JS	JavaScript
ORM	Object-Relational Mapping (Mapping Objet-Relationnel)
RBAC	Role-Based Access Control (Contrôle d'Accès Basé sur les Rôles)
i18n	Internationalization (Internationalisation)
LTR	Left-To-Right (Écriture de gauche à droite)
RTL	Right-To-Left (Écriture de droite à gauche)
HMR	Hot Module Replacement (Rechargement à chaud des modules)

Liste des figures

Figure 1: Organigramme de la Commune de Kasba Tadla.....	11
Figure 2: Diagramme de Gantt de la planification du projet.....	17
Figure 3: Schéma global de l'architecture du système.....	19
Figure 4: Diagramme de cas d'utilisation du système de réclamations	21
Figure 5: MCD du système de gestion des réclamations	22
Figure 6: L'historique des commits	29
Figure 7: Page Générale du site.....	37
Figure 8: Page Connexion	38
Figure 9: Page Inscription	39
Figure 10: Partie Citoyen - Listes des Réclamations.....	40
Figure 11: Partie Citoyen - Ajouter une nouvelle Réclamation.....	40
Figure 12: Profil de l'utilisateur.....	41
Figure 13: Partie Employé - Listes des Réclamations et ses Détails.....	42
Figure 14: Partie Admin - Tableau de bord	43
Figure 15: Partie Admin - Gestion des utilisateurs.....	43
Figure 16: Partie Admin - Gestion des employés.....	44
Figure 17: Diagramme de sequence	59
Figure 18: Diagramme de classes.....	60

Table des matières

Remerciements	1
Résumé	2
Table des Abréviations	3
Liste des figures	4
Table des matières	5
Introduction générale	8
Chapitre 1 : Présentation de la Commune Kasba Tadla	9
1. Introduction	10
2. Historique de la commune de Kasba Tadla :	10
3. Fiche technique :	10
4. L'organigramme de la commune :.....	11
5. Présentation des services de la Commune Urbaine de Kasba Tadla :	12
Chapitre 2 : Contexte et Analyse du Projet de stage	13
1. Problématique et motivations	14
2. Objectifs fonctionnels et non-fonctionnels.....	14
2.1. Objectifs fonctionnels.....	14
2.2. Objectifs non-fonctionnels	15
3. Périmètre et limites du projet	15
3.1. Périmètre couvert.....	15
3.2. Limites du projet.....	15
4. Analyse des utilisateurs.....	16
5. Étude comparative / Benchmark	16
6. Planification du projet	17
Chapitre 3 : Architecture et Conception du projet	18
1. Schéma global	19
2. Plan du site et navigation (User Flow)	19
3. Diagrammes UML utiles	20
4. Conception de la base de données	21
5. Choix techniques et justification	23
Chapitre 4 : Réalisation Technique du projet	24
1. Stack Frontend.....	25
2. Stack Backend	25
3. API consommée (Principaux Endpoints utilisés).....	27

4.	Gestion des rôles et sécurité	27
5.	Technologies et outils utilisées.....	27
5.1.	Outils utilisés	27
5.1.1.	Environnement de Développement.....	27
5.1.2.	Outils de Gestion de Projet.....	28
5.1.3.	Outils de Test	28
5.1.4.	Outils de Gestion de Version et de Collaboration.....	28
5.2.	Technologies utilisées	29
5.2.1.	Technologies Frontend.....	29
5.2.2.	Technologies Backend.....	30
Chapitre 5 : Implémentation par Fonctionnalité.....		31
1.	Parcours Citoyen	32
2.	Parcours employé de traitement	32
3.	Parcours chef de département.....	33
4.	Parcours Administrateur	33
5.	Internationalisation et accessibilité	34
6.	Gestion des médias (Upload des fichiers)	34
7.	Gestion des erreurs et notifications utilisateur	35
Chapitre 6 : Présentation des interfaces.....		36
1.	Introduction	37
2.	Page d'accueil	37
3.	Page de connexion.....	38
4.	Page d'inscription.....	39
5.	Page de dépôt de réclamation.....	40
6.	Page de profil d'un employé	41
7.	Tableau de bord employé	42
8.	Tableau de bord administrateur – Vue globale	43
9.	Tableau de bord administrateur – Gestion des utilisateurs	43
10.	Tableau de bord administrateur – Gestion des employés	44
Chapitre 7 : Tests et Validation		45
1.	Stratégie de test	46
2.	Outils de test.....	46
2.1.	Tests de l'API avec Postman	46
2.2.	Tests de l'interface utilisateur (UI)	46
3.	Résultats et corrections.....	47
3.1.	Sécurité et contrôle des accès	47

3.2.	Validation des données	47
3.3.	Gestion des erreurs et feedback utilisateur	47
Chapitre 8 : Déploiement et Exploitation.....		48
1.	Environnement de développement local	49
2.	Plan de déploiement	49
3.	Sécurité et sauvegardes envisagées	50
Chapitre 9 : Résultats, Limites et Perspectives.....		51
1.	Résultats obtenus.....	52
2.	Limites actuelles.....	52
3.	Perspectives d'évolution	52
Conclusion.....		53
Bilan.....		54
Bibliographie.....		56
Annexes		58
Annexe 1 : Diagrammes UML		59
Annexe 2 : Configuration de l'environnement.....		61
Annexe 3 : Extraits de code.....		63
Annexe 4 : Scripts de base de données.....		67

Introduction générale

Dans un contexte de transformation numérique croissante, les administrations publiques sont appelées à moderniser leurs services afin d'améliorer la qualité des prestations offertes aux citoyens. Parmi ces services, la gestion des réclamations constitue un élément essentiel de la relation entre l'administration et les usagers.

Cependant, dans plusieurs collectivités, ce processus reste basé sur des méthodes traditionnelles, entraînant des difficultés de suivi, des retards de traitement et une diminution de la satisfaction des usagers.

C'est dans ce cadre que s'inscrit ce projet de stage réalisé au sein de la **commune de Kasba Tadla**, ayant pour objectif la conception et la réalisation d'une application web dédiée à la gestion des réclamations. Cette solution vise à centraliser les demandes et à améliorer leur traitement de manière efficace et structurée.

Ce rapport présente les différentes étapes de réalisation du projet. Il est structuré comme suit :

- Le premier chapitre présente l'organisme d'accueil.
- Le deuxième chapitre décrit le contexte et l'analyse du projet.
- Le troisième chapitre est consacré à la conception et à l'architecture.
- Le quatrième chapitre traite de la réalisation technique.
- Les chapitres suivants présentent les fonctionnalités, les tests, le déploiement et les résultats obtenus.

Ce projet a permis d'appliquer les connaissances acquises durant la formation et de développer des compétences pratiques dans le domaine du développement web.

Chapitre 1 : Présentation de la Commune Kasba Tadla

1. Introduction

La commune constitue l'unité administrative de base la plus proche des citoyens. Elle joue un rôle essentiel dans la gestion des affaires locales, notamment dans les domaines de l'urbanisme, des services sociaux et de l'administration publique.

2. Historique de la commune de Kasba Tadla :

La création de la commune urbaine de Kasba-Tadla date de l'ère préfectorale en 1912.

Elle représentait l'administration du résident général. L'édifice public, situé au boulevard Mohamed V a été ensuite cédé gratuitement aux autorités marocaines.

En 1981, la commune urbaine, enregistrée selon le décret n° 212 a obtenu son statut de Pachalik après avoir été dépendante de la circonscription.

La construction du grand projet situé à l'est de l'ancienne municipalité a commencé en 2004. Il a fallu trois ans à sa réalisation.

La commune urbaine se compose de plusieurs bureaux et services, à sa tête, se trouve le bureau du conseil municipal formé du président et de cinq vices présidents.

3. Fiche technique :

Tableau 1: Fiche Technique de la commune de Kasba Tadla

Raison sociale	Commune Kasba Tadla
Adresse E-mail	Ckt.Serviceinfo@gmail.com
Fax	05 23 41 83 48
Tél	05 89 34 34 61
Activité	Gestion des affaires locales
Statut juridique	Administration publique
Dirigé par	Le conseil communal
Nombre de salariés	Environ 150

4. L'organigramme de la commune :

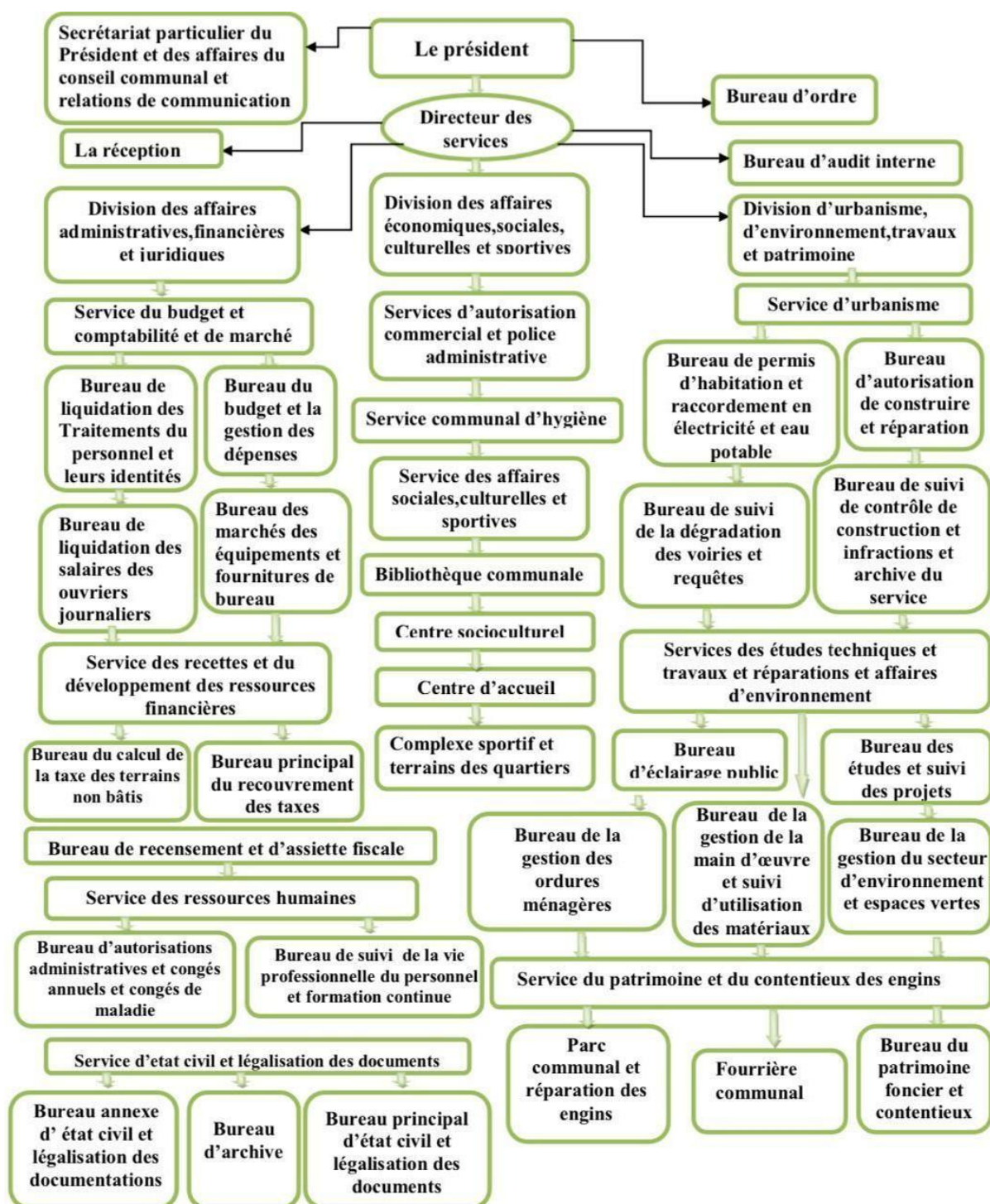


Figure 1: Organigramme de la Commune de Kasba Tadla

5. Présentation des services de la Commune Urbaine de Kasba Tadla :

Bureau du conseil municipal : composé des membres élus.

Secrétariat Général : a pour mission l'organisation, la gestion et la coordination entre les services de la commune urbaine, en collaboration avec le conseil municipal.

Secrétariat du président : Son rôle principal est de coordonner, harmoniser et affecter les tâches entre les sections, en plus elle assure :

- La réception, l'envoi et le classement du courrier émanant des autres services.
- La réponse aux appels téléphoniques.
- L'envoi des différents documents.

Bureau d'accueil et de renseignements : a pour fonction l'accueil et le renseignement des gens.

Service Bureau D'ordre : Sa fonction est la correspondance administrative (départs et arrivées).

Services d'État civil : a pour fonction la correspondance et la sauvegarde des registres de l'état civil notamment la légalisation des signatures, déclaration des naissances et décès.

Bureau de règlement de traitement de salaire : S'intéresse à la régulation de la situation financière des fonctionnaires et des agents et le suivi.

Bureau gestion administrative des dossiers personnels : A pour fonction de présentation de plusieurs documents (Contrat de travail, congé, contre visite, ordre de coupage du salaire, et reclassements du personnel).

Service des affaires financiers et économiques : établit les différents documents nécessaires à la gestion financière, il s'agit notamment du budget et compte administratif.

Bureau de la régie de recettes : Perçoit les taxes, redevances et droits dus à la commune urbaine dont les tarifs et taux sont fixés par l'arrêté municipal.

Service de patrimoine communal : maintient les dossiers relatifs aux biens communaux immobiliers (sommier de consistance).

Chapitre 2 : Contexte et Analyse du Projet de stage

1. Problématique et motivations

Dans de nombreuses organisations, publiques comme privées, la gestion des réclamations constitue un élément essentiel pour garantir la qualité des services et la satisfaction des utilisateurs. Cependant, cette gestion repose encore très souvent sur des méthodes traditionnelles, telles que les appels téléphoniques, les courriers électroniques ou des formulaires papier non centralisés.

Ces pratiques présentent plusieurs limites majeures, notamment un manque de traçabilité des demandes, une dispersion des informations, ainsi qu'une difficulté à assurer un suivi efficace et continu des réclamations. En conséquence, les délais de traitement peuvent s'allonger, certaines demandes peuvent être perdues ou négligées, et les utilisateurs ne disposent pas toujours d'une visibilité claire sur l'état d'avancement de leurs réclamations.

Face à ces problématiques, le projet « **Réclamation** » a été initié afin de proposer une solution numérique centralisée, capable de structurer et d'optimiser l'ensemble du processus de gestion des réclamations. L'objectif est de remplacer les méthodes classiques par un système moderne, fiable et accessible.

La motivation principale de ce projet réside dans la volonté d'améliorer la qualité du service offert aux utilisateurs, de renforcer leur satisfaction et de garantir une meilleure transparence dans le traitement des demandes. Par ailleurs, ce projet vise également à faciliter le travail des administrateurs en leur offrant des outils efficaces pour gérer, organiser et suivre les réclamations de manière optimale.

2. Objectifs fonctionnels et non-fonctionnels

2.1. Objectifs fonctionnels

Le système proposé vise à répondre aux besoins des différents acteurs à travers un ensemble de fonctionnalités principales.

Il doit permettre :

- ✓ L'inscription et l'authentification sécurisée des utilisateurs
- ✓ Le dépôt de réclamations avec possibilité d'ajouter des pièces jointes
- ✓ Le suivi en temps réel de l'état des réclamations
- ✓ La consultation de l'historique des demandes

- ✓ La gestion des réclamations par les employés (traitement, réponse, mise à jour)
- ✓ La gestion des utilisateurs et des rôles par l'administrateur
- ✓ L'organisation des réclamations selon plusieurs critères (statut, catégorie, priorité)

2.2. Objectifs non-fonctionnels

En complément des fonctionnalités, le système doit respecter un ensemble d'exigences non-fonctionnelles garantissant sa qualité, sa performance et sa sécurité :

Critère	Description
Sécurité	Authentification sécurisée, protection des données, contrôle des accès
Performance	Temps de réponse rapide même avec plusieurs utilisateurs
Ergonomie	Interface intuitive, simple et facile à utiliser
Accessibilité	Support bilingue (français et arabe) et adaptation mobile
Fiabilité	Stabilité du système et absence de perte de données
Évolutivité	Possibilité d'ajouter de nouvelles fonctionnalités

3. Périmètre et limites du projet

3.1. Périmètre couvert

Le projet couvre l'ensemble du cycle de vie d'une réclamation, incluant :

- La création de comptes utilisateurs
- Le dépôt et la gestion des réclamations
- Le suivi de leur état d'avancement
- L'administration et la supervision du système

Il permet également la centralisation des données et facilite la communication entre les différents acteurs.

3.2. Limites du projet

Malgré ses fonctionnalités, le système présente certaines limites :

- Absence de notifications en temps réel (SMS, notifications)

- Absence d'intégration avec d'autres systèmes externes
- Performances non testées à grande échelle
- Optimisation mobile partielle

Ces limites constituent des axes d'amélioration pour les futures versions.

4. Analyse des utilisateurs

Il y a de 4 d'utilisateurs de cette plateforme :

Type utilisateur	Rôle	Besoins
Citoyen	Soumettre et suivre les réclamations	Interface simple, suivi clair
Employé	Traiter les réclamations	Outils de gestion et organisation
Chef département	Gérer les employés et les réclamations de sa département	Outils de gestion et organisation
Administrateur	Gérer le système	Contrôle total et supervision

5. Étude comparative / Benchmark

Solution	Avantages	Limites
Emails	Facile à utiliser	Non structuré, difficile à suivre
Google Forms	Simple et rapide	Pas de suivi dynamique
Systèmes de ticketing	Fonctionnalités complètes	Complexité, non adapté aux communes
Solution proposée	Centralisée, adaptée, simple	Version initiale (améliorations possibles)

6. Planification du projet

Le diagramme de Gantt suivant présente l'ordonnancement chronologique des sept phases clés, de l'analyse initiale jusqu'à la finalisation du système.

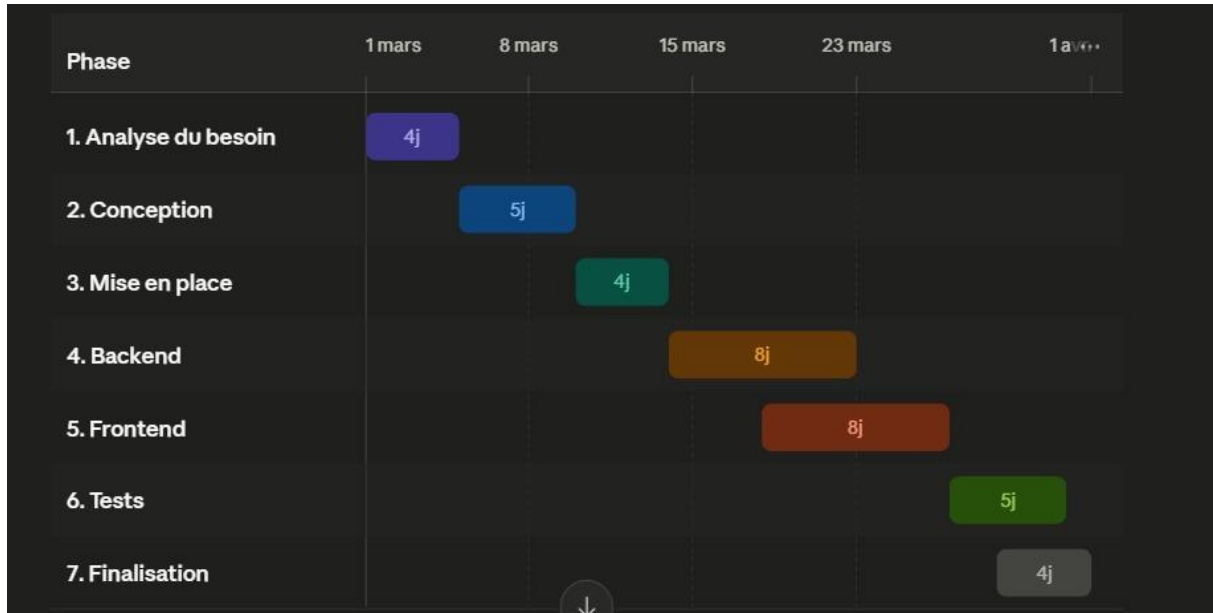


Figure 2: Diagramme de Gantt de la planification du projet

Chapitre 3 : Architecture et Conception du projet

1. Schéma global

L'architecture du système « **Réclamation** » repose sur une approche en trois couches (**architecture 3-tier**), permettant de séparer clairement les responsabilités et d'améliorer la maintenabilité du système.

Cette architecture est composée de :

- Une couche de présentation (**Frontend**)
- Une couche logique (**Backend / API**)
- Une couche de données (**Base de données**)

Cette organisation permet une meilleure évolutivité du système, en facilitant les mises à jour et l'intégration de nouvelles fonctionnalités sans impacter les autres composants.

La figure ci-dessous illustre l'interaction entre ces différentes couches.

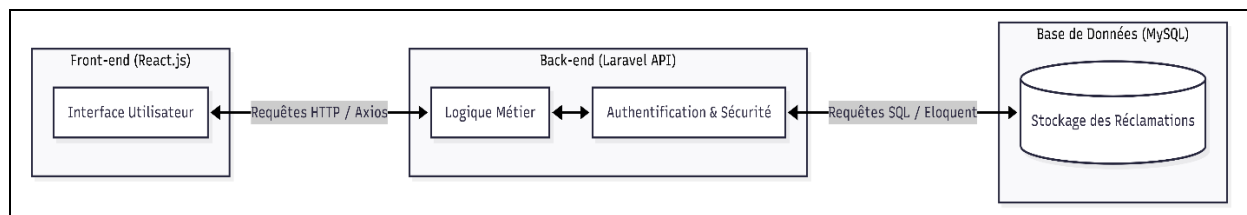


Figure 3: Schéma global de l'architecture du système

2. Plan du site et navigation (User Flow)

Le plan du site a été conçu afin de garantir une navigation intuitive et adaptée aux différents profils d'utilisateurs du système.

Pour un utilisateur non authentifié, l'accès se limite aux pages publiques telles que la connexion et l'inscription. Une fois authentifié, l'utilisateur est redirigé vers un espace spécifique en fonction de son rôle.

Le **citoyen** peut :

- Déposer une nouvelle réclamation
- Consulter l'historique de ses demandes
- Suivre l'état d'avancement (en attente, en cours, complétée, rejetée)

L'**employé** dispose d'un espace lui permettant de :

- Consulter les réclamations assignées

- Mettre à jour leur statut
- Interagir avec les citoyens

Le **chef de département** a pour rôle de :

- Superviser les réclamations
- Assigner les tâches aux départements
- Suivre l'avancement global d'une réclamation
- Gérer les employés de son département

L'**administrateur** possède un accès complet au système, incluant :

- La gestion des utilisateurs
- La gestion des départements
- La supervision globale des réclamations
- La modification des données techniques de la commune

Le parcours utilisateur (User Flow) est structuré de manière à réduire la complexité et à améliorer l'efficacité des interactions, tout en garantissant une expérience fluide et cohérente.

3. Diagrammes UML utiles

Dans le cadre de la conception du projet, l'utilisation des diagrammes UML s'avère essentielle pour modéliser le système et représenter de manière claire les interactions entre ses différents composants.

Le diagramme de cas d'utilisation (Use Case) permet d'identifier les principaux acteurs du système, notamment l'utilisateur, l'employé de traitement et l'administrateur, ainsi que les différentes fonctionnalités auxquelles ils ont accès. Ce diagramme met en évidence des actions telles que l'inscription, la connexion, la soumission d'une réclamation, la consultation de l'historique, la gestion des demandes et l'administration des utilisateurs. Il constitue un outil fondamental pour la compréhension des besoins fonctionnels.

Le diagramme de séquence, quant à lui, permet de représenter le déroulement chronologique des interactions entre les composants du système. Par exemple, dans le scénario de soumission d'une réclamation, l'utilisateur saisit ses données via le frontend, celles-ci sont envoyées au backend sous forme de requête API, puis traitées et validées avant d'être

enregistrées dans la base de données. Une réponse est ensuite renvoyée vers le frontend pour informer l'utilisateur du résultat de l'opération.

Ces diagrammes jouent un rôle clé dans la phase de conception, en facilitant la communication entre les différents intervenants du projet et en assurant une meilleure compréhension du fonctionnement global du système.

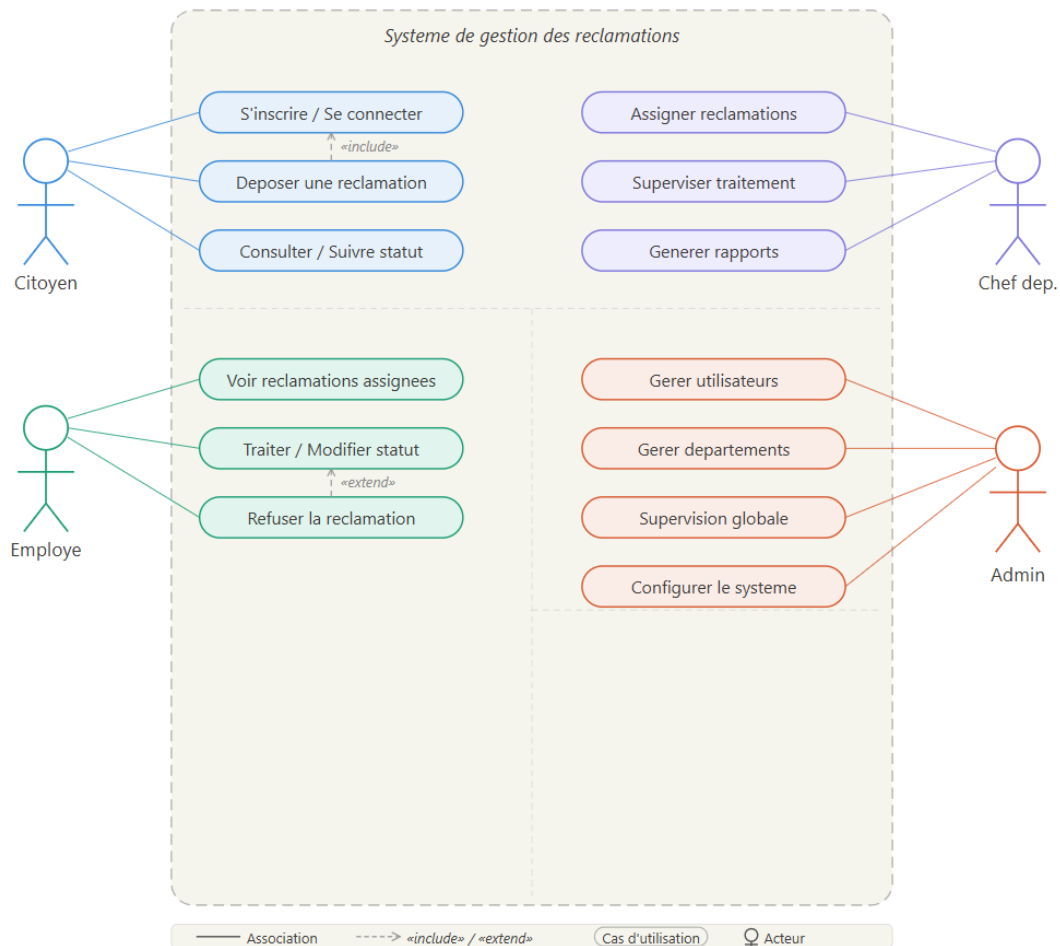


Figure 4: Diagramme de cas d'utilisation du système de réclamations

4. Conception de la base de données

La conception de la base de données est illustrée à l'aide du diagramme MCD présenté ci-dessous, permettant de visualiser les entités, leurs relations ainsi que les cardinalités.

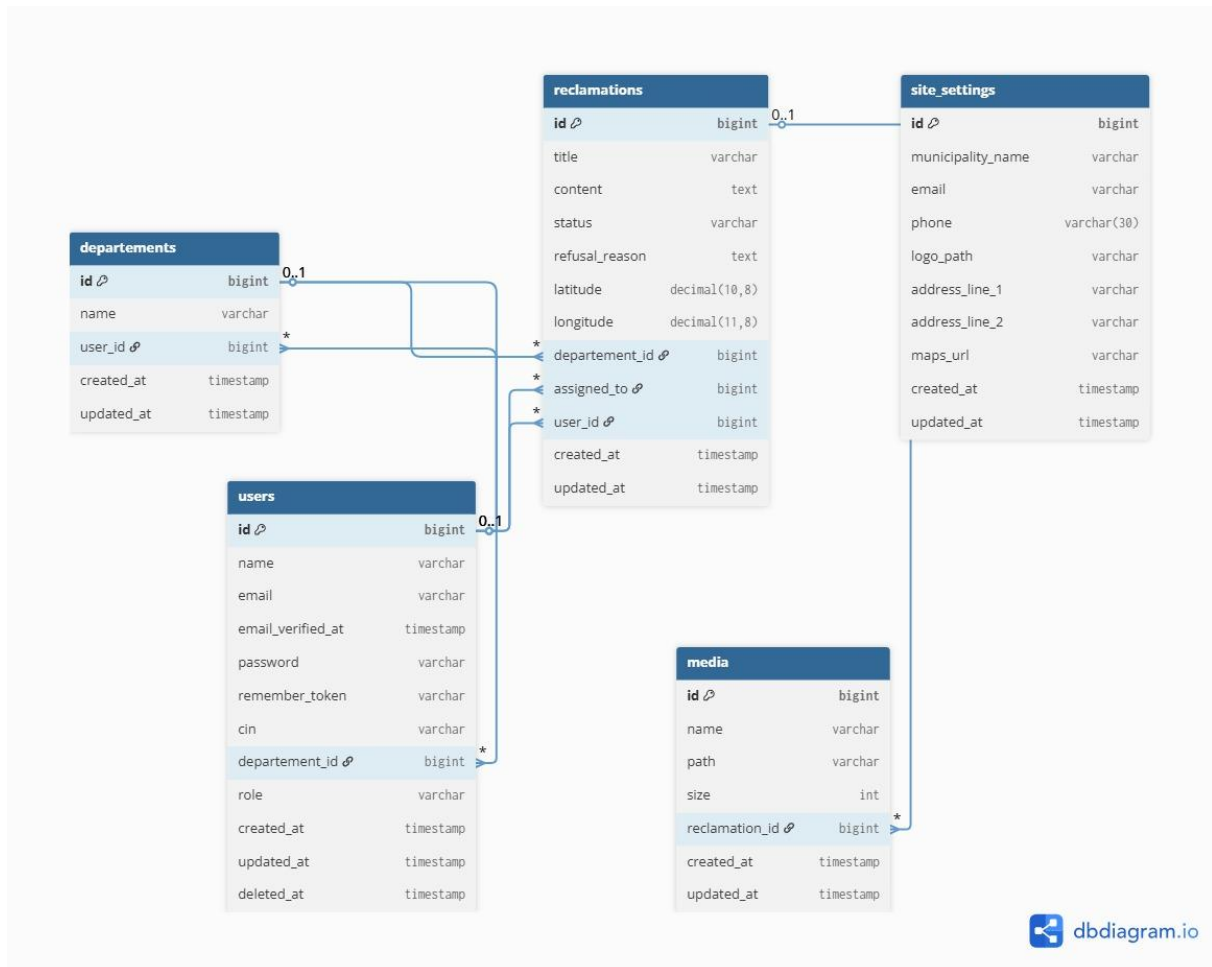


Figure 5: MCD du système de gestion des réclamations

Les principales entités du système sont :

- **users** : contient les informations des comptes ainsi que les rôles (admin, chef département, employé, citoyen).
- **reclamations** : représente les demandes soumises par les citoyens.
- **departements** : permet d'organiser les services de la commune.
- **media** : définit la preuve de chaque réclamation envoyée par le citoyen.
- **site_settings** : définit les informations techniques de la commune.

Chaque réclamation est associée à :

- Un utilisateur (citoyen)
- Un département
- Un employé assigné
- Des coordonnées géographiques (latitude, longitude)

Des clés primaires et étrangères ont été mises en place afin d'assurer les relations entre les tables. Des contraintes de validation garantissent également la qualité et la cohérence des données. Cette structure permet une gestion efficace des informations et facilite les opérations de recherche, de mise à jour et d'évolution du système.

5. Choix techniques et justification

Les choix techniques du projet ont été effectués en fonction de plusieurs critères : performance, maintenabilité, sécurité et évolutivité.

- **Frontend (React avec Vite) :** Permet de développer une interface moderne, rapide et basée sur des composants réutilisables. Cela améliore l'expérience utilisateur et facilite la maintenance du code.
- **Backend (Framework Laravel) :** Choisi en raison de sa robustesse, de sa structure MVC et de ses fonctionnalités intégrées. De plus, l'utilisation de **Laravel Sanctum** permet de sécuriser l'authentification via API, ce qui est essentiel dans une architecture séparée frontend/backend.
- **Architecture (API REST) :** Assure une communication claire et standardisée entre les différentes couches du système, tout en offrant la possibilité d'intégrer d'autres clients (mobile, applications externes) à l'avenir.
- **Base de données relationnelle :** Retenue pour sa fiabilité et sa capacité à gérer des relations complexes entre les entités.
- **Outils de développement (Git, environnement local, outils de test) :** Leur utilisation a permis d'assurer une meilleure organisation du projet, un suivi des versions efficace et une amélioration continue de la qualité du code.

Chapitre 4 : Réalisation Technique du projet

1. Stack Frontend

Le frontend est développé avec **React (via Vite)**, offrant une architecture orientée composants réutilisables, une compilation rapide et des performances optimisées en production.

Technologie	Rôle
React + Vite	Structure en composants, rendering performant, HMR en développement
React Router	Routing côté client (SPA) — navigation sans rechargement de page
Axios	Client HTTP pour la consommation de l'API REST, gestion des intercepteurs et des erreurs
Tailwind CSS	Stylisation utilitaire, design responsive et cohérent
Framer Motion	Animations déclaratives pour les transitions et micro-interactions

L'application est segmentée en espaces distincts selon les rôles. Le contrôle d'accès aux routes est géré côté client en fonction du rôle retourné par l'API après authentification.

2. Stack Backend

Le backend est développé avec **Laravel**, exposant une **API REST** structurée selon le modèle **MVC** et sécurisée via **Laravel Sanctum**.

Architecture MVC :

- **Modèles** — mapping relationnel et interactions avec la base de données (Eloquent ORM).
- **Contrôleurs** — traitement des requêtes, logique métier, retour des réponses JSON.
- **Routes** — définies dans **routes/api.php**, qui assurent la communication avec le frontend.

Authentification — Laravel Sanctum : Toutes les routes protégées requièrent un **token Bearer** valide (middleware : auth : sanctum). Le token est émis à la connexion et invalidé à la déconnexion.

Middlewares :

- auth : sanctum — vérifie la validité du token sur chaque requête protégée.

```
Route::get('/user', function (Request $request) {
    return $request->user()->load('departement');
})->middleware('auth:sanctum');
```

- Contrôle du rôle utilisateur au niveau des contrôleurs via **\$request->user()->role** et de l'accès aux routes sensibles :

```
Route::middleware(['auth:sanctum'])->group(function () {
    Route::apiResource('departements', DepartementController::class);
    Route::apiResource('reclamations', ReclamationController::class);
    Route::patch('reclamations/{reclamation}/refuse', [ReclamationController::class, 'refuse']);
    Route::apiResource('users', UserController::class);
    Route::post('site-settings', [SiteSettingController::class, 'update']);
});
```

Cette architecture backend permet de construire une API robuste, sécurisée et facilement extensible.

Laravel Sanctum :

Laravel Sanctum est un système d'authentification léger conçu spécifiquement pour les SPA (Single Page Applications) et les API REST.

Comment ça fonctionne dans ton projet :

Lors de la connexion, Sanctum génère un **token personnel** stocké en base de données dans la table `personal_access_tokens`. Ce token est renvoyé au frontend, qui le stocke et l'envoie dans le header de chaque requête protégée : `Authorization: Bearer 3|abc123xyz...`

À chaque requête, le middleware `auth:sanctum` intercepte la requête, vérifie que le token existe en base, n'est pas expiré et correspond à un utilisateur valide. Si le token est invalide ou absent → réponse 401 Unauthorized. À la déconnexion, le token est supprimé de la base — il ne peut plus être réutilisé.

Pourquoi Sanctum et pas JSON Web Token(JWT) ?

	Sanctum	JWT
Stockage	Base de données	Autonome (signé)
Révocation	Instantanée	Impossible sans blacklist
Complexité	Simple	Plus complexe
Adapté SPA	Oui	Oui

Sanctum est préféré ici car il permet de **révoquer un token immédiatement** (deconnexion réel), ce que JWT ne permet pas nativement — un JWT reste valide jusqu'à expiration même après déconnexion.

3. API consommée (Principaux Endpoints utilisés)

Le projet expose une **API REST** consommée par le frontend via **Axios**. Les routes publiques couvrent l'authentification (register, login, logout). L'ensemble des autres **endpoints** gestion des réclamations, des utilisateurs et des départements est protégé par le middleware auth : sanctum, qui exige un **token Bearer** valide à chaque requête. L'accès aux routes d'administration est restreint côté contrôleur selon le rôle de l'utilisateur authentifié.

4. Gestion des rôles et sécurité

Le système implémente un contrôle d'accès basé sur les rôles (**RBAC**), où chaque utilisateur n'accède qu'aux ressources correspondant à son rôle.

Mécanismes de sécurité mis en place :

Mécanisme	Implémentation
Authentification	Laravel Sanctum — tokens par session, révocables
Autorisation	Vérification du rôle dans chaque contrôleur
Protection des routes	Middleware sur toutes les routes privées
Validation des entrées	Form Requests Laravel — rejet des données invalides avant traitement
Protection contre les injections	Eloquent ORM — requêtes préparées, pas de SQL brut

5. Technologies et outils utilisés

5.1. Outils utilisés

5.1.1. Environnement de Développement

Visual Studio Code Éditeur de code extensible pour le développement et le débogage du code.	
--	---

	Xampp Outil de développement incluant Apache, MySQL et PHP pour tester l'application localement
---	--

5.1.2. Outils de Gestion de Projet

GanttProject Logiciel open-source de gestion de projet utilisé pour la planification et l'ordonnancement des tâches.	
---	---

5.1.3. Outils de Test

Postman Outil de développement utilisé pour tester et valider les endpoints de l'API en simulant des requêtes HTTP.	
--	---

5.1.4. Outils de Gestion de Version et de Collaboration

GitHub GitHub est une plateforme de développement collaboratif qui permet de gérer le code source et de suivre les modifications apportées au projet.	
	Git Système de gestion de versions décentralisé permettant de suivre l'évolution du code source.

Historique des commits : La figure ci-dessous illustre l'historique des contributions enregistrées tout au long du développement du projet.

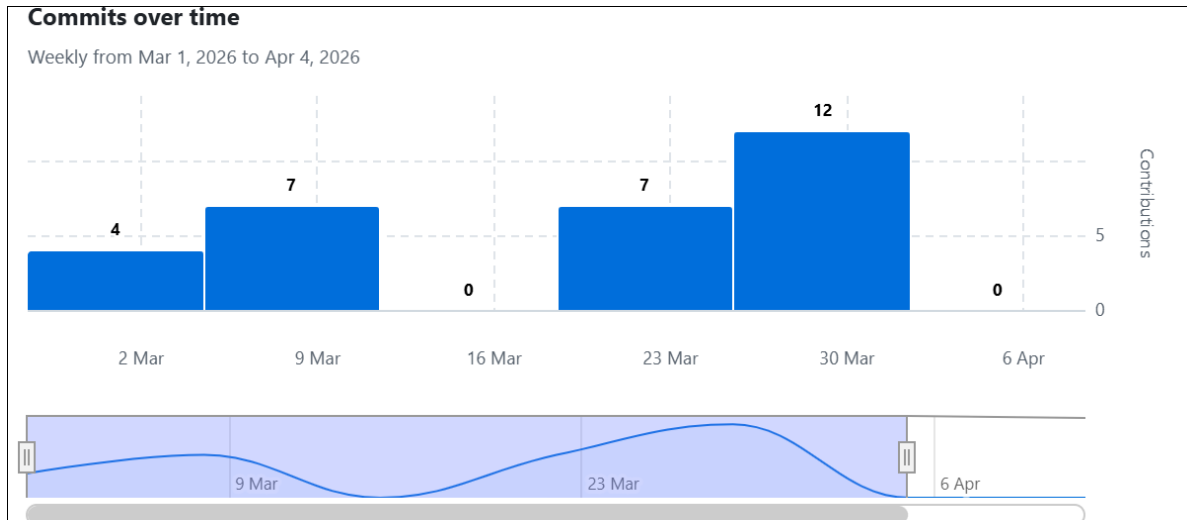


Figure 6: L'historique des commits

5.2. Technologies utilisées

5.2.1. Technologies Frontend

HTML5 Pour structurer les pages web avec des éléments sémantiques.	
	CSS3 Pour styliser les pages web et créer des mises en page réactives.
Tailwind CSS Framework CSS pour faciliter la création de designs réactifs.	
	JavaScript Pour rendre les pages web interactives.

React

Bibliothèque JavaScript moderne utilisée pour la création d'interfaces utilisateur interactives et dynamiques. Elle repose sur une architecture basée sur des composants réutilisables, permettant de construire des applications web performantes et évolutives.



5.2.2. Technologies Backend

Laravel

Framework PHP moderne et élégant, conçu pour faciliter le développement d'applications web robustes. Il propose une syntaxe expressive et des outils intégrés puissants pour gérer efficacement les bases de données, le routage, l'authentification et les migrations de données.



MySQL

Système de gestion de bases de données relationnelles pour stocker et gérer les données du système de gestion des réclamations

Chapitre 5 : Implémentation par Fonctionnalité

1. Parcours Citoyen

Le parcours citoyen correspond aux différentes étapes suivies par l'utilisateur principal lors de son interaction avec la plateforme. Il a été conçu pour garantir une expérience **simple, intuitive et accessible**, même pour les utilisateurs peu familiers avec les outils numériques.

Après authentification, le citoyen accède à son espace personnel, à partir duquel il peut soumettre une nouvelle réclamation via un formulaire structuré comprenant les informations essentielles (objet, description, catégorie, pièces jointes éventuelles).

L'utilisateur dispose également d'un espace lui permettant de consulter l'historique de ses réclamations. Chaque demande est associée à un statut (en attente, en cours de traitement, traitée ou rejetée), offrant une **visibilité claire sur son évolution**.

Enfin, le citoyen peut consulter les réponses fournies par les services concernés, ce qui améliore la transparence et facilite la communication avec l'administration.

2. Parcours employé de traitement

L'employé de traitement est chargé de la gestion opérationnelle des réclamations. Il dispose d'un tableau de bord dédié regroupant les demandes qui lui sont assignées.

À partir de cet espace, il peut :

- consulter les détails des réclamations
- analyser les informations fournies
- mettre à jour le statut de la réclamation
- ajouter des réponses ou commentaires en cas de refus d'une réclamation

Il peut également interagir avec le citoyen en cas de besoin (demande d'informations complémentaires).

Ce parcours permet d'assurer un **traitement structuré, rapide et organisé** des réclamations.

3. Parcours chef de département

Le chef de département joue un rôle intermédiaire entre l'administration et les employés de traitement. Il assure la **coordination et la supervision des réclamations** au sein de son département.

Depuis son tableau de bord, il peut :

- consulter l'ensemble des réclamations liées à son département
- suivre l'avancement global des réclamations
- intervenir en cas de mauvaise orientation de réclamation
- gérer les membres de son département

Ce rôle est essentiel pour garantir une **répartition efficace des tâches** et un suivi optimal des réclamations.

4. Parcours Administrateur

L'administrateur assure la gestion globale du système. Il dispose d'un tableau de bord centralisé offrant une vue d'ensemble sur :

- les réclamations
- les utilisateurs
- les départements
- les paramètres du système

Ses principales responsabilités incluent :

- gestion des comptes utilisateurs (création, modification, suppression)
- gestion des rôles et des permissions
- supervision des réclamations
- maintenance des données globales du système

Ce parcours permet un **contrôle complet et une gestion stratégique** de la plateforme.

5. Internationalisation et accessibilité

Afin de garantir une plateforme inclusive, évolutive et adaptée à son contexte d'utilisation, le projet intègre nativement des standards d'internationalisation et d'ergonomie :

- **Internationalisation (i18n) :**
 - **Architecture Bilingue :** Implémentation d'un système de traduction dynamique permettant l'ajout et la gestion centralisée des langues sans modification du code source.
 - **Gestion bidirectionnelle (RTL / LTR) :** Adaptation automatique de l'interface (layout, marges, typographie) pour supporter les directions de lecture LTR (français) et RTL (arabe).
- **Accessibilité et Expérience Utilisateur (UI/UX) :** Interface intuitive basée sur une structure claire et un design cohérent. Standardisation des composants (boutons, formulaires) pour faciliter l'usage, avec une conception responsive assurant une expérience optimale sur tous les supports.

6. Gestion des médias (Upload des fichiers)

La plateforme permet aux utilisateurs d'ajouter des pièces jointes (images, vidéos ...etc.) lors de la soumission d'une réclamation.

Cette fonctionnalité permet :

- d'enrichir les informations fournies
- de mieux illustrer les problèmes signalés
- d'améliorer la qualité du traitement

Les fichiers sont :

- validés (type et taille)
- stockés de manière sécurisée
- accessibles uniquement aux utilisateurs autorisés

7. Gestion des erreurs et notifications utilisateur

La gestion des erreurs et des notifications est essentielle pour assurer une bonne expérience utilisateur.

En cas d'erreur, le système affiche des messages clairs indiquant :

- la nature du problème
- les actions à corriger

Par ailleurs, des notifications sont utilisées pour informer l'utilisateur des actions réussies (création, modification, réponse).

Ces mécanismes permettent :

- d'améliorer la compréhension du système
- de réduire les erreurs
- de renforcer la confiance de l'utilisateur

Chapitre 6 : Présentation des interfaces

1. Introduction

Ce chapitre décrit les principales interfaces de la plateforme, en mettant l'accent sur leur organisation, leur lisibilité et leur facilité d'utilisation. Chaque page a un rôle précis dans le parcours utilisateur et assure une navigation fluide.

2. Page d'accueil

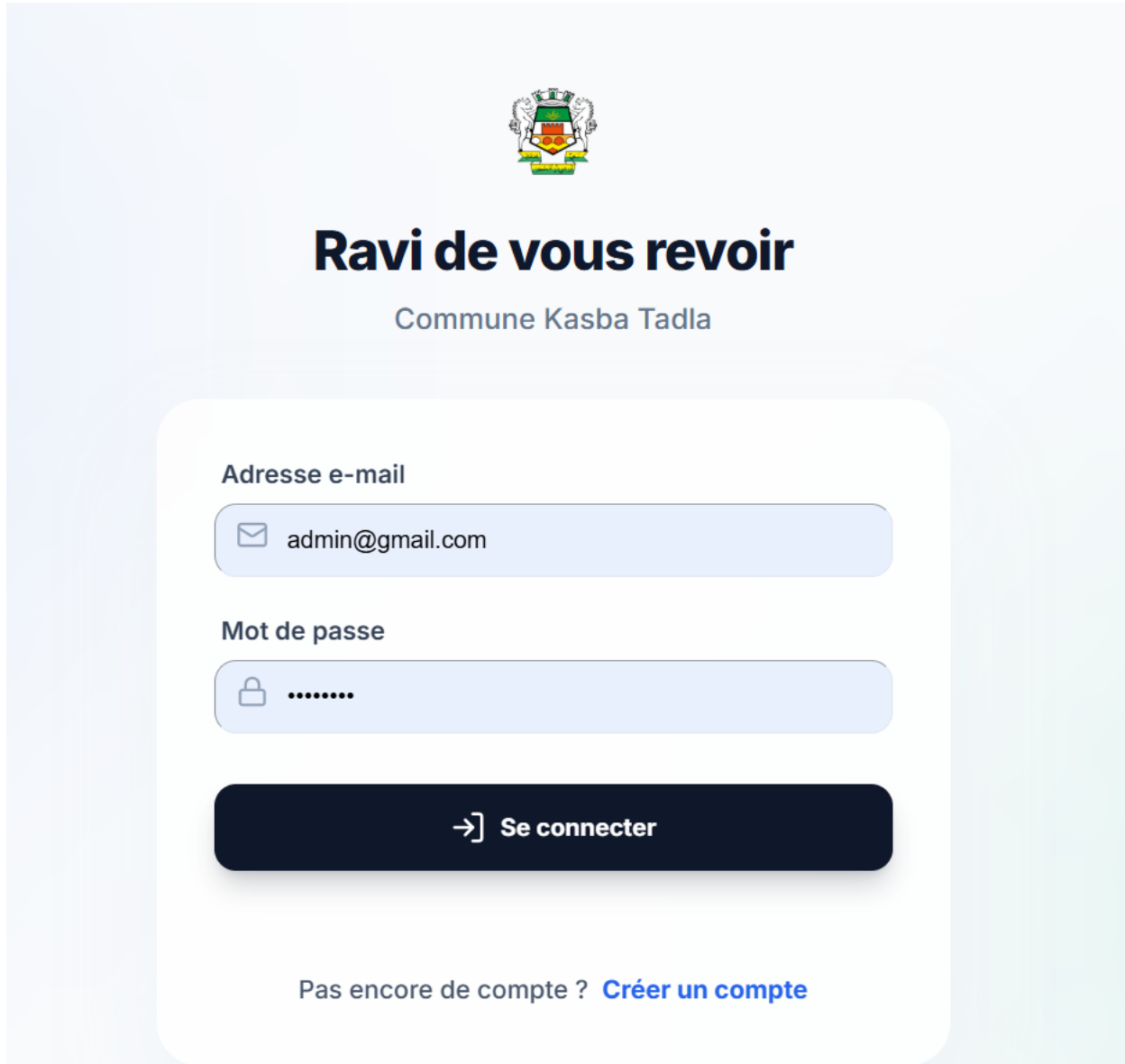
Point d'entrée de la plateforme, elle présente le projet et oriente l'utilisateur vers les actions principales : connexion, inscription et accès aux informations essentielles.



Figure 7: Page Générale du site

3. Page de connexion

Permet aux utilisateurs existants d'accéder à leur espace personnel via e-mail et mot de passe, avec validation des erreurs pour garantir sécurité et simplicité.



The image shows a login interface for the 'Commune Kasba Tadla'. At the top center is a coat of arms. Below it, the text 'Ravi de vous revoir' is displayed in a large, bold, dark blue font, followed by 'Commune Kasba Tadla' in a smaller, grey font. The login form is a white rounded rectangle with a subtle shadow. It contains two input fields: 'Adresse e-mail' with an email icon and the text 'admin@gmail.com', and 'Mot de passe' with a lock icon and masked characters '.....'. A dark blue button with a right arrow icon and the text 'Se connecter' is positioned below the password field. At the bottom of the form, the text 'Pas encore de compte ?' is followed by a blue link 'Créer un compte'.



Ravi de vous revoir

Commune Kasba Tadla

Adresse e-mail

 admin@gmail.com

Mot de passe



 **Se connecter**

Pas encore de compte ? [Créer un compte](#)

Figure 8: Page Connexion

4. Page d'inscription

Destinée aux nouveaux utilisateurs, elle collecte les informations nécessaires à la création de compte (nom, e-mail, mot de passe) et assure leur validité.

Créer un compte
Rejoignez l'espace citoyen officiel

Nom complet
Rayan AlAnsari

Adresse e-mail
rayan@gmail.com

Mot de passe
.....

Confirmer le mot de passe
.....

S'inscrire

Figure 9: Page Inscription

5. Page de dépôt de réclamation

Interface clé pour soumettre une réclamation avec objet, description, catégorie et pièces justificatives, guidant l'utilisateur et limitant les erreurs.

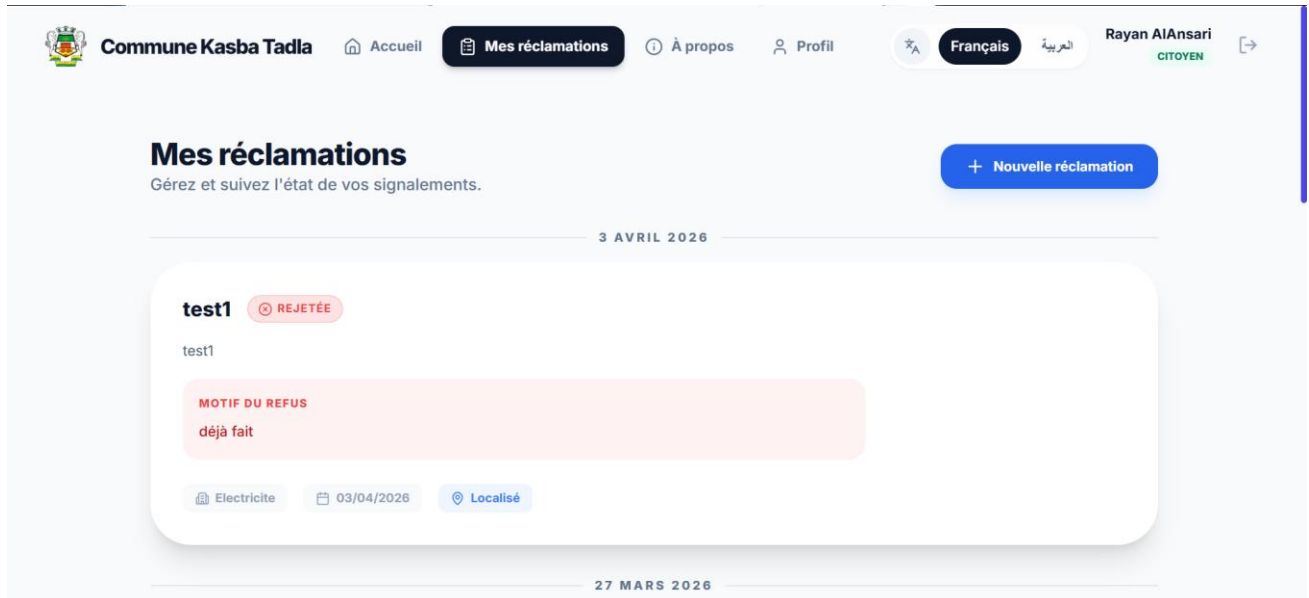


Figure 10: Partie Citoyen - Listes des Réclamations

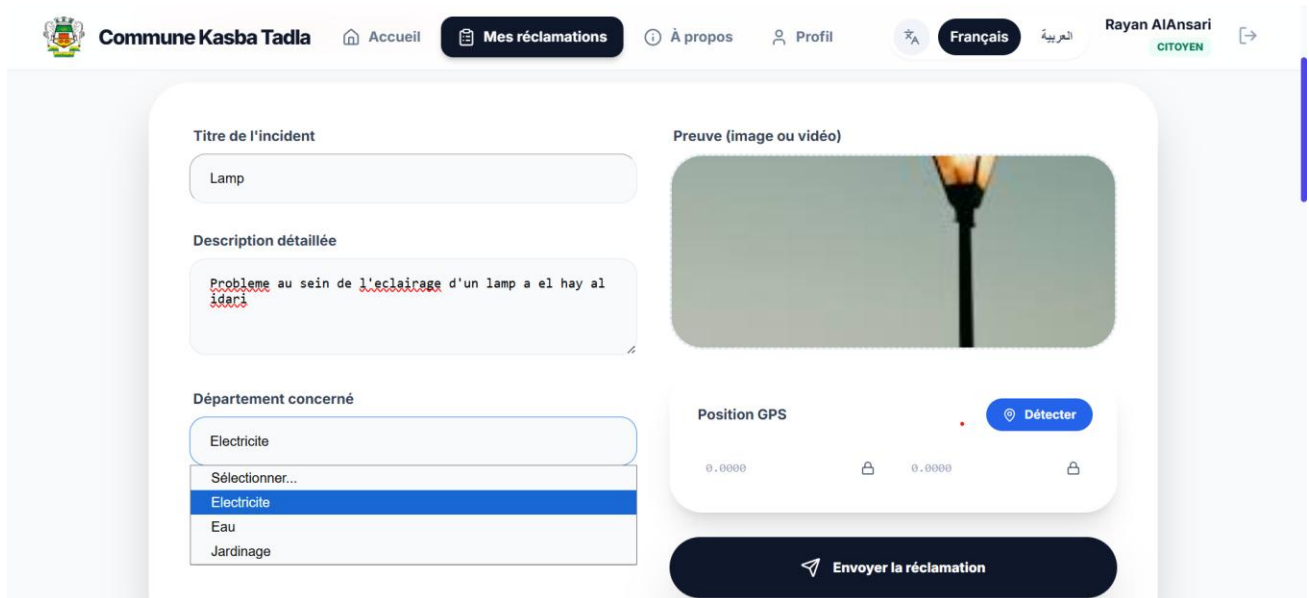


Figure 11: Partie Citoyen - Ajouter une nouvelle Réclamation

6. Page de profil d'un employé

Présente les informations sur l'employé ou le service (rôle, coordonnées, spécialisation), offrant transparence et visibilité.

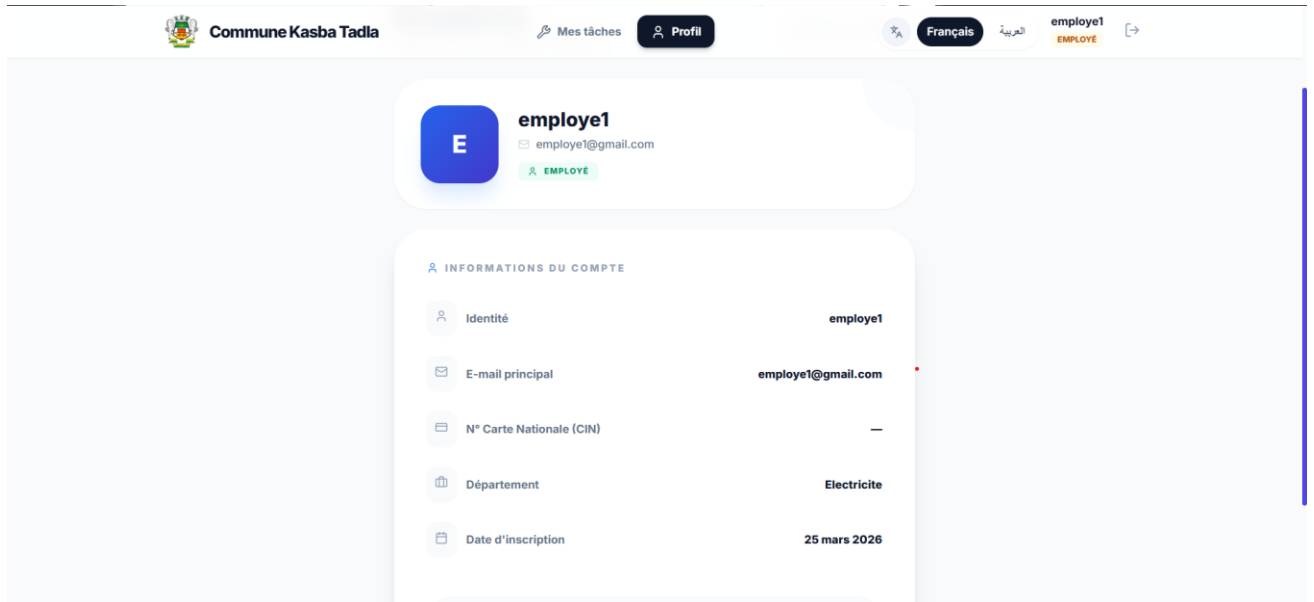


Figure 12: Profil de l'utilisateur

7. Tableau de bord employé

Permet de suivre les réclamations, consulter les détails, mettre à jour leur état et interagir avec les utilisateurs.

The screenshot displays the 'Commune Kasba Tadla' employee dashboard. At the top, there's a header with the commune logo, 'Commune Kasba Tadla', 'Mes tâches', 'Profil', language options ('Français', 'العربية'), and a user profile 'employe1 EMPLOYE'. The main section is titled 'Mes tâches assignées' with a subtitle '3 réclamation(s) dans votre département'. Below this, three summary cards show: '0 À TRAITER' (yellow clock icon), '3 RÉSOLUES' (green checkmark icon), and '1 REFUSÉES' (red X icon). A date separator '3 AVRIL 2026' is shown. The first task card, 'test1', is marked 'REJETÉE' (red X icon) and dated '3 avril 2026'. It includes a 'DESCRIPTION' field with 'test1' and a 'MOTIF DU REFUS' field with 'déjà fait'. Below is a map titled 'LOCALISATION' showing a street view with a red pin. A yellow banner at the bottom of the map states 'Cette réclamation est déjà prise par un autre employe.' A date separator '26 MARS 2026' follows. The next task card, 'eer', is marked 'TERMINÉE' (green checkmark icon) and dated '26 mars 2026' with '1 Fichier(s)'. A final date separator '25 MARS 2026' is shown. The last task card, 'dftyujk', is marked 'TERMINÉE' (green checkmark icon) and dated '25 mars 2026'.

Figure 13: Partie Employé - Listes des Réclamations et ses Détails

8. Tableau de bord administrateur – Vue globale

Fournit une vision globale de la plateforme : nombre de réclamations, des utilisateurs, des départements, des employés pour superviser efficacement le système.

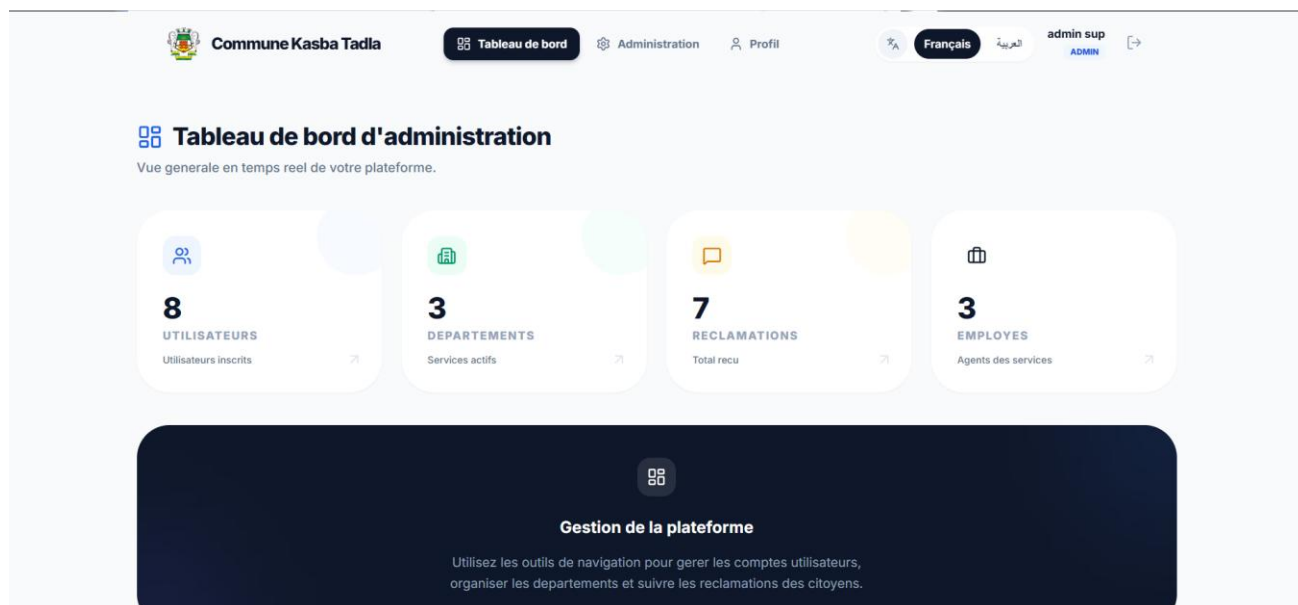


Figure 14: Partie Admin - Tableau de bord

9. Tableau de bord administrateur – Gestion des utilisateurs

Permet de consulter et organiser les comptes utilisateurs, contrôler les accès et vérifier les profils.

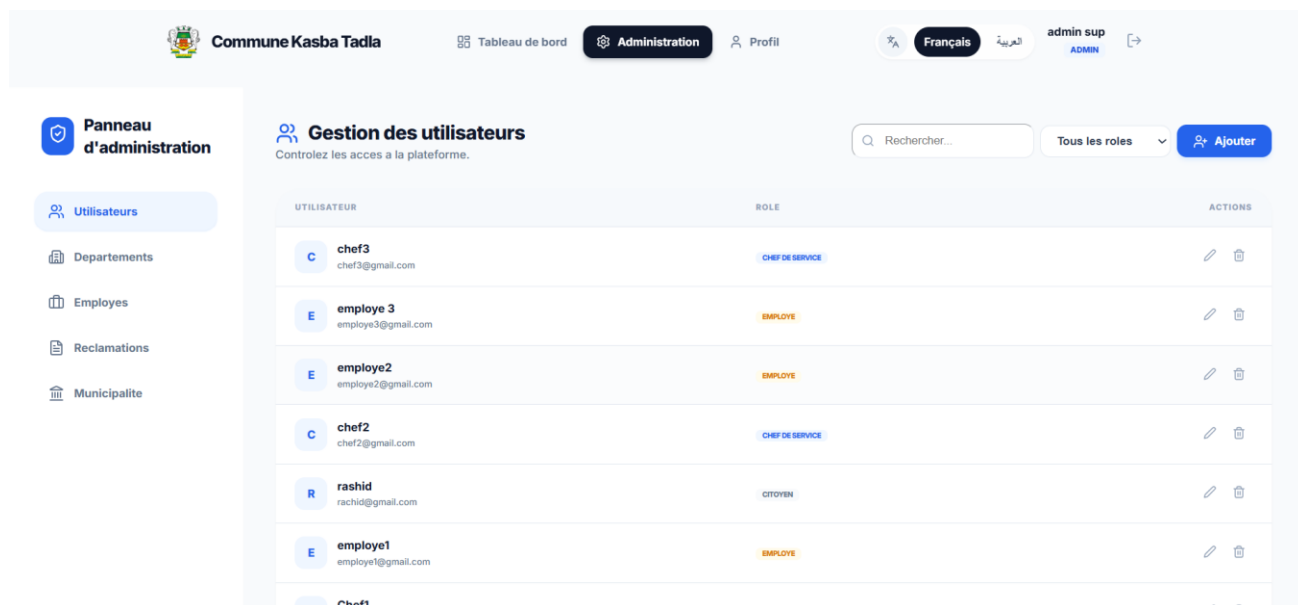


Figure 15: Partie Admin - Gestion des utilisateurs

10. Tableau de bord administrateur – Gestion des employés

Supervise les comptes des employés, leur activité et la répartition des responsabilités pour assurer un traitement efficace des réclamations.

Commune Kasba Tadla | Tableau de bord | Administration | Profil | Français | العربية | admin sup ADMIN

Panneau d'administration

- Utilisateurs
- Departements
- Employes**
- Reclamations
- Municipalite

Employes
3 membres enregistrés

Rechercher... | Tous les departements | Ajouter

ID	EMPLOYE	DEPARTEMENT	ACTIONS
#8	employe 3 employe3@gmail.com	ELECTRICITE	
#7	employe2 employe2@gmail.com	EAU	
#4	employe1 employe1@gmail.com	ELECTRICITE	

PAGE 1 SUR 1 - 3 RESULTATS | < Precedent | Suivant >

Figure 16: Partie Admin - Gestion des employés

Chapitre 7 : Tests et Validation

1. Stratégie de test

La phase de test est essentielle pour s'assurer du bon fonctionnement de l'ensemble des fonctionnalités de la plateforme. La stratégie adoptée repose sur des scénarios de tests manuels reproduisant les actions réelles des utilisateurs selon leurs rôles (citoyen, employé de traitement et administrateur).

Les principaux objectifs étaient :

- Vérifier la création de compte et l'authentification sécurisée
- Tester le dépôt et le suivi des réclamations
- Contrôler la gestion des rôles et des redirections
- Vérifier les fonctionnalités administratives

Une attention particulière a été accordée à la gestion des rôles afin de garantir que chaque utilisateur accède uniquement aux fonctionnalités autorisées selon son profil.

2. Outils de test

Pour garantir la qualité du système, deux types de tests ont été réalisés :

2.1. Tests de l'API avec Postman

- Vérification des end-points backend : création de compte, authentification, dépôt et suivi des réclamations, gestion des utilisateurs.
- Contrôle de la validité des réponses et des formats attendus.
- Détection des erreurs logiques ou de validation et correction pour améliorer la fiabilité du backend

2.2. Tests de l'interface utilisateur (UI)

- Vérification de l'affichage correct de toutes les pages et de la fluidité de navigation.
- Contrôle des formulaires, des boutons, des messages d'erreur et des notifications.
- Repérage des anomalies visuelles ou ergonomiques pouvant nuire à l'expérience utilisateur.

3. Résultats et corrections

Les tests ont permis d'identifier plusieurs points à améliorer et de corriger des problèmes techniques et fonctionnels, renforçant ainsi la **stabilité, la sécurité et la qualité globale** de la plateforme.

3.1. Sécurité et contrôle des accès

- Vérification que certaines pages ou fonctionnalités ne sont accessibles qu'aux utilisateurs autorisés.
- Ajustement des permissions et renforcement de l'authentification pour éviter tout accès non autorisé.

3.2. Validation des données

- Amélioration des formulaires pour empêcher l'envoi de données incomplètes ou incohérentes.
- Réduction des erreurs lors du traitement des réclamations et amélioration de la qualité des informations enregistrées.

3.3. Gestion des erreurs et feedback utilisateur

- Mise en place de messages d'erreur clairs et de confirmations d'action réussie.
- Gestion améliorée des échecs pour rendre l'application plus compréhensible et intuitive.

Chapitre 8 : Déploiement et Exploitation

1. Environnement de développement local

Le projet a été développé dans un environnement local structuré, dont les principales composantes sont présentées dans le tableau suivant :

Composant	Outil / Version
Système d'exploitation	Windows 11
Éditeur de code	Visual Studio Code
Serveur local	XAMPP (Apache + MySQL)
Runtime backend	PHP 8.2
Framework backend	Laravel 12
Runtime frontend	Node.js 20.x
Framework frontend	React 18 + Vite 5
Base de données	MySQL 8.0
Versioning	Git + GitHub
Tests API	Postman

2. Plan de déploiement

Même si le projet est actuellement en phase de développement, un **plan de déploiement** est envisagé pour une mise en production future :

Déploiement du frontend

- Publication de la version de production sur une plateforme d'hébergement web.
- Application accessible, rapide et stable pour les utilisateurs finaux.

Déploiement du backend

- Installation sur un serveur capable d'exécuter l'application et de gérer les requêtes API.
- Configuration des variables d'environnement, connexion à la base de données et gestion des autorisations.

Déploiement de la base de données

- Installation sur un serveur fiable pour garantir disponibilité, sécurité et cohérence des données.
- Création de la structure relationnelle et configuration des accès.
- Mise en place de sauvegardes régulières et optimisation des performances.

3. Sécurité et sauvegardes envisagées

La sécurité et la sauvegarde des données sont essentielles pour l'exploitation réelle de la plateforme :

Sécurité

- Authentification des utilisateurs et gestion des rôles.
- Protection des routes sensibles et validation des données échangées.
- En production : utilisation du protocole HTTPS, configuration sécurisée des serveurs et surveillance continue.

Sauvegardes et maintenance

- Sauvegardes régulières et automatiques de la base de données pour prévenir toute perte d'information.
- Stratégies de restauration fiables.
- Maintenance continue pour corriger les bugs, mettre à jour les composants et assurer la stabilité du système à long terme.

Chapitre 9 : Résultats, Limites et Perspectives

1. Résultats obtenus

Le projet **Réclamation** a permis de développer une **plateforme web centralisée** pour la gestion numérique des réclamations. Les résultats obtenus montrent que les objectifs principaux ont été atteints :

- **Pour les utilisateurs** : possibilité de soumettre des réclamations, de suivre leur évolution et de communiquer avec l'administration à travers une interface unique et intuitive.
- **Pour la gestion interne** : organisation des réclamations, suivi de leur état, gestion des comptes utilisateurs et amélioration de la traçabilité des actions.

Cette réalisation constitue une **avancée concrète vers la digitalisation et l'optimisation des services**, en simplifiant le processus de gestion et en centralisant les informations essentielles.

2. Limites actuelles

Malgré les résultats obtenus, certaines limites subsistent :

- Absence de fonctionnalités avancées (notifications temps réel, analyse statistique, automatisation)
- Optimisation limitée pour une forte montée en charge
- Expérience mobile perfectible

Ces limites n'affectent pas la valeur du projet, mais indiquent que la plateforme constitue **une base évolutive**, prête à être enrichie progressivement.

3. Perspectives d'évolution

Plusieurs améliorations peuvent être envisagées :

- Intégration de notifications (email, SMS)
- Mise en place d'un module statistique avancé
- Priorisation intelligente des réclamations
- Amélioration de la version mobile (responsive design)
- Renforcement de la sécurité
- Intégration avec d'autres systèmes

Ces évolutions permettront d'augmenter la **valeur et l'impact du système**.

Conclusion

Au terme de ce projet de stage, la plateforme **Réclamation** s'impose comme une réponse concrète et structurée aux limites des méthodes traditionnelles de gestion des réclamations au sein de la Commune de Kasba Tadla. En centralisant l'ensemble des demandes au sein d'un système unique, l'application améliore significativement la traçabilité, réduit les délais de traitement et renforce la communication entre les citoyens et l'administration.

Sur le plan technique, ce projet a mobilisé un ensemble de technologies modernes — React, Laravel, MySQL — organisées selon une architecture en trois couches garantissant la séparation des responsabilités, la sécurité et l'évolutivité du système. Les phases de conception, de développement et de test ont permis de livrer une solution fiable, couvrant l'intégralité du cycle de vie d'une réclamation.

Sur le plan personnel, cette expérience a été particulièrement enrichissante. Elle a permis de confronter les connaissances acquises en formation aux exigences réelles d'un environnement professionnel, et de développer une approche méthodique et rigoureuse du développement web.

Bilan

Ce stage au sein de la Commune de Kasba Tadla a constitué une immersion professionnelle complète, alliant conception technique, travail en équipe et adaptation aux contraintes d'un projet réel.

Problèmes rencontrés et solutions apportées

Au cours du développement, plusieurs difficultés techniques ont été rencontrées. L'intégration du système d'authentification dans une architecture SPA séparant le frontend et le backend a nécessité une compréhension approfondie du fonctionnement de Laravel Sanctum, notamment en ce qui concerne la gestion des jetons et des mécanismes de sécurité tels que les cookies CSRF.

La mise en place du support bilingue (français/arabe), notamment la gestion des interfaces en mode RTL/LTR, a également constitué un défi au niveau de la conception des interfaces avec Tailwind CSS. Ce problème a été résolu en adaptant les layouts et en utilisant les fonctionnalités du framework permettant de gérer la direction des éléments.

Ces difficultés ont été surmontées grâce à la consultation de la documentation officielle, aux recherches personnelles ainsi qu'aux échanges avec les encadrants.

Compétences développées

Sur le plan technique, ce stage m'a permis de consolider mes compétences en développement web full stack, notamment avec React, Laravel, les API REST et MySQL. J'ai également acquis de bonnes pratiques professionnelles, telles que la gestion de versions avec Git et la réalisation de tests d'API avec Postman.

Sur le plan humain, cette expérience a contribué au développement de mon esprit d'équipe, de ma communication professionnelle et de ma capacité d'adaptation à un environnement de travail réel.

En effet, le projet a été réalisé en collaboration avec deux autres membres, ce qui m'a permis d'apprendre à travailler efficacement en équipe. Nous avons réparti les tâches selon les compétences de chacun, assuré un suivi régulier de l'avancement et communiqué de manière continue pour garantir la cohérence du projet.

Par exemple, lors des phases de développement et de correction, nous échangeons régulièrement pour résoudre les problèmes techniques, partager des solutions et prendre des

décisions communes. Cette collaboration m'a permis de comprendre l'importance de la coordination, de l'écoute et du partage des responsabilités dans la réussite d'un projet.

Perspectives et ouverture

Cette expérience ouvre plusieurs perspectives d'évolution. À court terme, la plateforme pourrait être enrichie et déployée officiellement au sein de la commune dans le cadre d'une future période de stage ou d'un projet de fin d'études.

À plus long terme, ce projet s'inscrit dans la dynamique de digitalisation des services publics au Maroc, un domaine en pleine expansion offrant de nombreuses opportunités professionnelles.

Par ailleurs, des programmes de formation avancée, de stages spécialisés ou encore des opportunités de bourses dans le domaine du développement web et de l'e-gouvernance pourraient constituer une suite logique pour approfondir ces compétences.

Bibliographie

Bibliographie

- Documentation officielle de Laravel : <https://laravel.com>
- Documentation officielle de React : <https://react.dev>
- Documentation officielle d'Axios : <https://axios-http.com>
- Documentation officielle de Tailwind CSS : <https://tailwindcss.com>
- Documentation officielle de MySQL : <https://dev.mysql.com>
- Documentation officielle de Postman : <https://www.postman.com>
- Documentation officielle de GitHub : <https://docs.github.com>
- Documentation officielle de Framer Motion : <https://www.framer.com/motion>
- Outil de modélisation de bases de données : <https://dbdiagram.io>
- Laracasts — tutoriels Laravel : <https://laracasts.com>
- MDN Web Docs — référence JavaScript et web : <https://developer.mozilla.org>
- Supports pédagogiques internes — cours de développement web full stack, ISTA Kasba Tadla, 2025

Annexes

Annexe 1 : Diagrammes UML

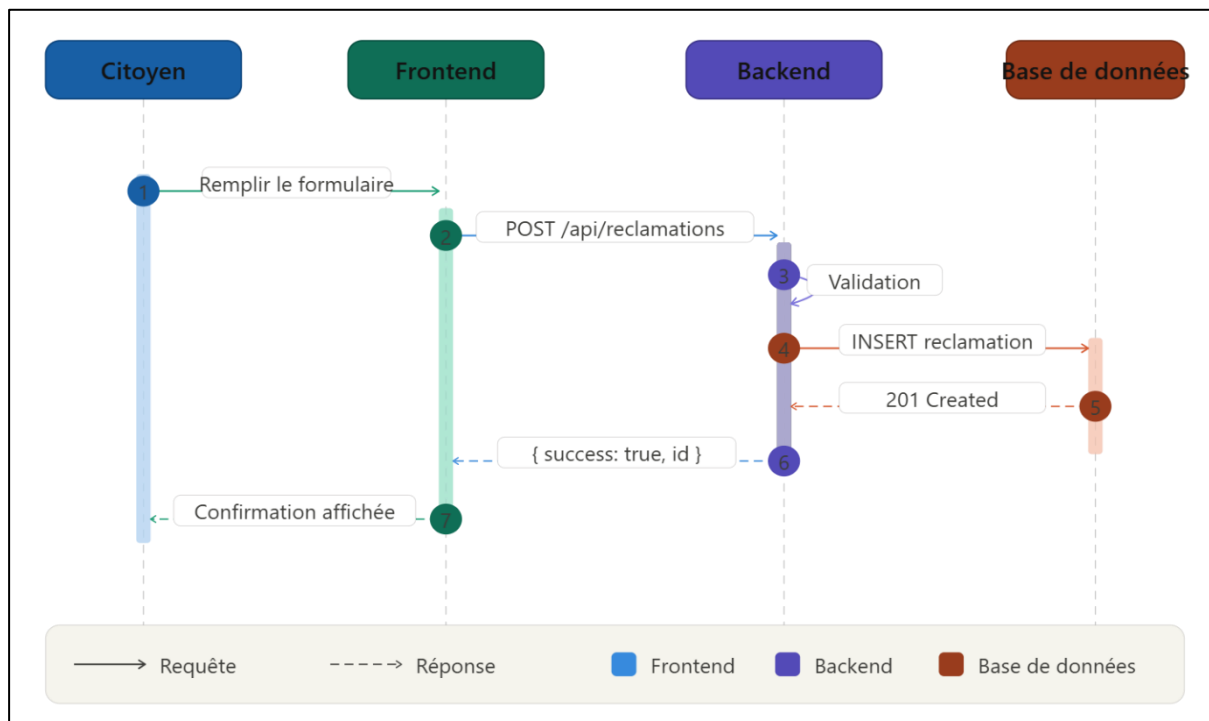


Figure 17: Diagramme de sequence

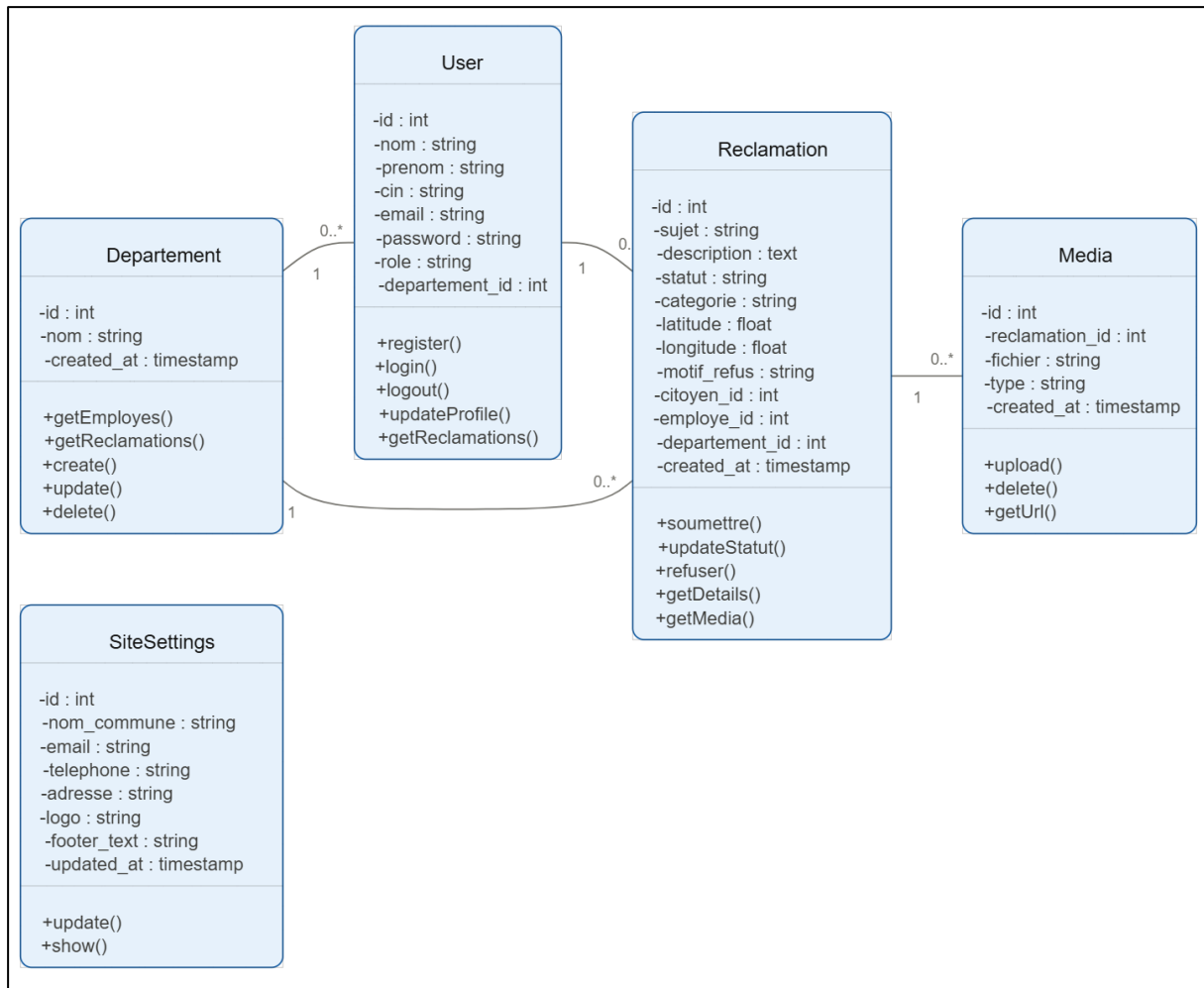


Figure 18: Diagramme de classes

Annexe 2 : Configuration de l'environnement

Configuration backend :

```
Import to Postman
1 APP_NAME=Reclamtion
2 APP_ENV=local
3 APP_KEY=base64:w2pcZaQnN/BkzaSlqN5IOZ/TnMwf8p3FrItR1VzNw10=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 DB_CONNECTION=mysql
8 DB_HOST=127.0.0.1
9 DB_PORT=3306
10 DB_DATABASE=reclamation
11 DB_USERNAME=root
12 DB_PASSWORD=
13
14 APP_LOCALE=en
15 APP_FALLBACK_LOCALE=en
16 APP_FAKER_LOCALE=en_US
17
18 VITE_APP_NAME="${APP_NAME}"
19 MAIL_FROM_ADDRESS="hello@example.com"
20 MAIL_FROM_NAME="${APP_NAME}"
```

Configuration frontend :

```
frontend > package.json > ...
1  {
2    "name": "frontend",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "lint": "eslint .",
10     "preview": "vite preview"
11   },
12   "dependencies": {
13     "axios": "^1.13.6",
14     "i18next": "^25.10.10",
15     "leaflet": "^1.9.4",
16     "lucide-react": "^1.7.0",
17     "react": "^19.2.4",
18     "react-dom": "^19.2.4",
19     "react-i18next": "^15.7.4",
20     "react-leaflet": "^5.0.0",
21     "react-router-dom": "^7.13.1"
22   },
23   "devDependencies": {
24     "@eslint/js": "^9.39.4",
25     "@types/react": "^19.2.14",
26     "@types/react-dom": "^19.2.3",
27     "@vitejs/plugin-react": "^6.0.0",
28     "autoprefixer": "^10.4.20",
29     "eslint": "^9.39.4",
30     "eslint-plugin-react-hooks": "^7.0.1",
31     "eslint-plugin-react-refresh": "^0.5.2",
32     "globals": "^17.4.0",
33     "postcss": "^8.5.6",
34     "tailwindcss": "^3.4.13",
35     "vite": "^8.0.0"
36   }
}
```

Annexe 3 : Extraits de code

Exemple de contrôleur d'authentification en Laravel

```

 9  class AuthController extends Controller
10  {
11      public function register(Request $request){
12          $validated=$request->validate([
13              'name'      => 'required|string|max:255',
14              'email'     => 'required|string|email|max:255|unique:users',
15              'password' => 'required|string|min:8|confirmed',
16              'cin'       => 'nullable|string|max:8',
17          ]);
18          $validated['role'] = 'citoyen';
19          $user = User::create($validated);
20          $user->load('departement');
21
22          $token = $user->createToken('auth_token')->plainTextToken;
23          return response()->json([
24              'message' => 'User registered successfully',
25              'user'    => $user,
26              'token'   => $token,
27          ], 201);
28      }
29
30      public function login(Request $request)
31      {
32          $credentials = $request->validate([
33              'email' => ['required', 'email'],
34              'password'=> 'required|string|min:8'
35          ]);
36          if (!Auth::attempt($credentials)) {
37              return response()->json([
38                  'message' => 'Invalid credentials'
39              ], 401);
40          }
41          $user = $request->user()->load('departement');
42          $token = $user->createToken('loginToken')->plainTextToken;
43          return response()->json([
44              'message' => 'success',
45              'user'    => $user,
46              'token'   => $token
47          ], 200);
48      }
49
50      public function logout(Request $request){
51          $request->user()->currentAccessToken()->delete();
52          return response()->json([
53              'message'=> 'Logout with success.'
54          ]);
55      }
56  }

```



```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\User;
6  use App\Models\Departement;
7  use Illuminate\Http\Request;
8  use Illuminate\Support\Facades\Hash;
9
10 class UserController extends Controller
11 {
12     public function index(Request $request)
13     {
14
15         $query = User::with('departement');
16         $query->where('id', '!=', $request->user()->id);
17         if ($request->has('search') && !empty($request->search)) {
18             $searchTerm = $request->search;
19             $query->where(function($q) use ($searchTerm) {
20                 $q->where('name', 'like', "%{$searchTerm}%")
21                     ->orWhere('email', 'like', "%{$searchTerm}%");
22             });
23         }
24         if ($request->has('role') && !empty($request->role)) {
25             $query->where('role', $request->role);
26         }
27         $users = $query->latest()->paginate(15);
28         return response()->json($users);
29     }
```

Requêtes API du Backend

```
1  <?php
2
3  use App\Http\Controllers\DepartementController;
4  use App\Http\Controllers\AuthController;
5  use App\Http\Controllers\ReclamationController;
6  use App\Http\Controllers\SiteSettingController;
7  use Illuminate\Http\Request;
8  use App\Http\Controllers\UserController;
9  use Illuminate\Support\Facades\Route;
10 use Illuminate\Support\Facades\Auth;
11
12 Route::get('/user', function (Request $request) {
13     return $request->user()->load('departement');
14 }->middleware('auth:sanctum');
15
16 Route::post('auth/register', [AuthController::class, 'register']);
17 Route::post('auth/login', [AuthController::class, 'login']);
18 Route::post('auth/logout', [AuthController::class, 'logout']->middleware('auth:sanctum');
19 Route::get('site-settings', [SiteSettingController::class, 'show']);
20
21 Route::middleware(['auth:sanctum']->group(function () {
22     Route::apiResource('departements', DepartementController::class);
23     Route::apiResource('reclamations', ReclamationController::class);
24     Route::patch('reclamations/{reclamation}/refuse', [ReclamationController::class, 'refuse']);
25     Route::apiResource('users', UserController::class);
26     Route::post('site-settings', [SiteSettingController::class, 'update']);
27 });
```

Requêtes API du Frontend

```

1  import axios from 'axios';
2
3  const api = axios.create({
4    baseURL: 'http://localhost:8000/api',
5    headers: {
6      'Content-Type': 'application/json',
7      'Accept': 'application/json',
8    },
9  });
10
11
12  api.interceptors.request.use((config) => {
13    const token = localStorage.getItem('token');
14    if (token) {
15      config.headers.Authorization = `Bearer ${token}`;
16    }
17    return config;
18  });
19
20  api.interceptors.response.use(
21    (response) => response,
22    (error) => {
23      if (error.response?.status === 401) {
24        localStorage.removeItem('token');
25        window.dispatchEvent(new Event('auth:logout'));
26      }
27      return Promise.reject(error);
28    }
29  );
30
31  // — Auth —
32  export const authService = {
33    register: (data) => api.post('/auth/register', data),
34    login: (data) => api.post('/auth/login', data),
35    logout: () => api.post('/auth/logout'),
36    getCurrentUser: () => api.get('/user'),
37  };
38
39  // — Reclamations —
40  export const reclamationService = {
41    getAll: (params) => api.get('/reclamations', { params }),
42    getMy: (userId, params) => api.get('/reclamations', { params: { ...params, user_id: userId } }),
43    getById: (id) => api.get(`/reclamations/${id}`),
44    create: (data) => api.post('/reclamations', data),
45    createWithMedia: (data) => api.post('/reclamations', data, {
46      headers: { 'Content-Type': 'multipart/form-data' }
47    }),
48    update: (id, data) => api.put(`/reclamations/${id}`, data),
49    refuse: (id, data) => api.patch(`/reclamations/${id}/refuse`, data),
50    delete: (id) => api.delete(`/reclamations/${id}`),
51  };
52
53  // — Departements —
54  export const departementService = {
55    getAll: (params) => api.get('/departements', { params }),
56    getById: (id) => api.get(`/departements/${id}`),
57    create: (data) => api.post('/departements', data),
58    update: (id, data) => api.put(`/departements/${id}`, data),
59    delete: (id) => api.delete(`/departements/${id}`),
60  };
61
62  // — Users —
63  export const userService = {
64    getAll: (params) => api.get('/users', { params }),
65    getById: (id) => api.get(`/users/${id}`),
66    create: (data) => api.post('/users', data),
67    update: (id, data) => api.put(`/users/${id}`, data),
68    delete: (id) => api.delete(`/users/${id}`),
69  };
70
71  export const siteSettingsService = {
72    get: () => api.get('/site-settings'),
73    update: (data) => api.post('/site-settings', data, {
74      headers: { 'Content-Type': 'multipart/form-data' },
75    }),
76  };
77
78  export default api;

```

Annexe 4 : Scripts de base de données

Création des tables :

Les scripts suivants présentent la structure des principales tables de la base de données du projet sont implémentées à l'aide des migrations du Framework Laravel.

➤ **Table users**

La table users est fournie par défaut par Laravel. Elle a été enrichie par l'ajout de nouvelles colonnes afin de répondre aux besoins du projet, notamment :

- cin : numéro de la carte d'identité nationale
- role : citoyen, employé, chef de département ou administrateur

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('users', function (Blueprint $table) {
12             $table->id();
13             $table->string('name');
14             $table->string('email')->unique();
15             $table->timestamp('email_verified_at')->nullable();
16             $table->string('password');
17             $table->rememberToken();
18             $table->timestamps();
19         });
20
21         Schema::create('password_reset_tokens', function (Blueprint $table) {
22             $table->string('email')->primary();
23             $table->string('token');
24             $table->timestamp('created_at')->nullable();
25         });
26
27         Schema::create('sessions', function (Blueprint $table) {
28             $table->string('id')->primary();
29             $table->foreignId('user_id')->nullable()->index();
30             $table->string('ip_address', 45)->nullable();
31             $table->text('user_agent')->nullable();
32             $table->longText('payload');
33             $table->integer('last_activity')->index();
34         });
35     }
36     public function down(): void
37     {
38         Schema::dropIfExists('users');
39         Schema::dropIfExists('password_reset_tokens');
40         Schema::dropIfExists('sessions');
41     }
42 };
```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table("users", function (Blueprint $table) {
            $table->string("cin")->nullable();
            $table->foreignId("departement_id")->nullable()->constrained("departements");
            $table->enum("role", ["citoyen", "employe", "chef_dep", "admin"])->default("citoyen");
        });
    }

    public function down(): void
    {
        //
    }
};

```

➤ Table departements

```

3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('departements', function (Blueprint $table) {
12             $table->id();
13             $table->string('name');
14             $table->foreignId('user_id')->nullable()->constrained('users')->nullOnDelete();
15             $table->timestamps();
16         });
17     }
18
19     public function down(): void
20     {
21         Schema::dropIfExists('departements');
22     }
23 };
24

```

➤ Table reclamations

```

3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     public function up(): void
10     {
11         Schema::create('reclamations', function (Blueprint $table) {
12             $table->id();
13             $table->string('title');
14             $table->text(column: 'content')->nullable();
15             $table->string('status')->default('en_attent');
16             $table->decimal('latitude', 10, 8);
17             $table->decimal('longitude', 11, 8);
18             // departement_id
19             $table->foreignId('departement_id')->constrained('departements');
20             $table->foreignId('user_id')->constrained();
21             $table->timestamps();
22         });
23     }
24     public function down(): void
25     {
26         Schema::dropIfExists('reclamations');
27     }
28 };
29

```

➤ Mise à jour de la table reclamations

Ajout du motif de refus d'une réclamation et de la responsable du traitement de la réclamation afin d'améliorer le suivi des réclamations.

```

1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     public function up(): void
10     {
11         Schema::table('reclamations', function (Blueprint $table) {
12             $table->text('refusal_reason')->nullable()->after('status');
13         });
14     }
15
16     public function down(): void
17     {
18         Schema::table('reclamations', function (Blueprint $table) {
19             $table->dropColumn('refusal_reason');
20         });
21     }
22 };

```

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::table('reclamations', function (Blueprint $table) {
12             $table->foreignId('assigned_to')
13                 ->nullable()
14                 ->after('departement_id')
15                 ->constrained('users')
16                 ->nullOnDelete();
17         });
18     }
19
20     public function down(): void
21     {
22         Schema::table('reclamations', function (Blueprint $table) {
23             $table->dropConstrainedForeignId('assigned_to');
24         });
25     }
26 };

```

➤ Table media

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('media', function (Blueprint $table) {
12             $table->id();
13             $table->string('name');
14             $table->string('path');
15             $table->integer('size');
16             $table->foreignId('reclamation_id')->constrained('reclamations');
17             $table->timestamps();
18         });
19     }
20
21     public function down(): void
22     {
23         Schema::dropIfExists('media');
24     }
25 };

```


➤ Table site_settings

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('site_settings', function (Blueprint $table) {
12             $table->id();
13             $table->string('municipality_name');
14             $table->string('email');
15             $table->string('phone', 30);
16             $table->string('logo_path')->nullable();
17             $table->timestamps();
18         });
19     }
20
21     public function down(): void
22     {
23         Schema::dropIfExists('site_settings');
24     }
25 };
```

➤ Mise à jour de la table site_settings

Des colonnes supplémentaires ont été ajoutées afin de permettre l’affichage des informations dans le footer du site.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::table('site_settings', function (Blueprint $table) {
12             $table->string('address_line_1')->nullable()->after('phone');
13             $table->string('address_line_2')->nullable()->after('address_line_1');
14             $table->string('maps_url')->nullable()->after('address_line_2');
15         });
16     }
17
18     public function down(): void
19     {
20         Schema::table('site_settings', function (Blueprint $table) {
21             $table->dropColumn(['address_line_1', 'address_line_2', 'maps_url']);
22         });
23     }
24 };
```